Coprocesadores : GPU, TPU y FPGA

Hermilo

17 de Febrero de 2026

Coprocesadores : ¿Por qué?

- Funcionalidad especializada.
  - Funciones no encontradas en CPU genérico.
  - Más rápidas que un CPU genérico.
- Diseño simplificado.
  - Menos costo que un CPU genérico.
  - No puede ejecutar sistema operativo.
  - Requiere de un CPU anfitrión.
- Más poder de cómputo a menor costo.
- Más compacto.
  - Menos espacio.
  - Menos energía.

Coprocesadores : Lo dificil.

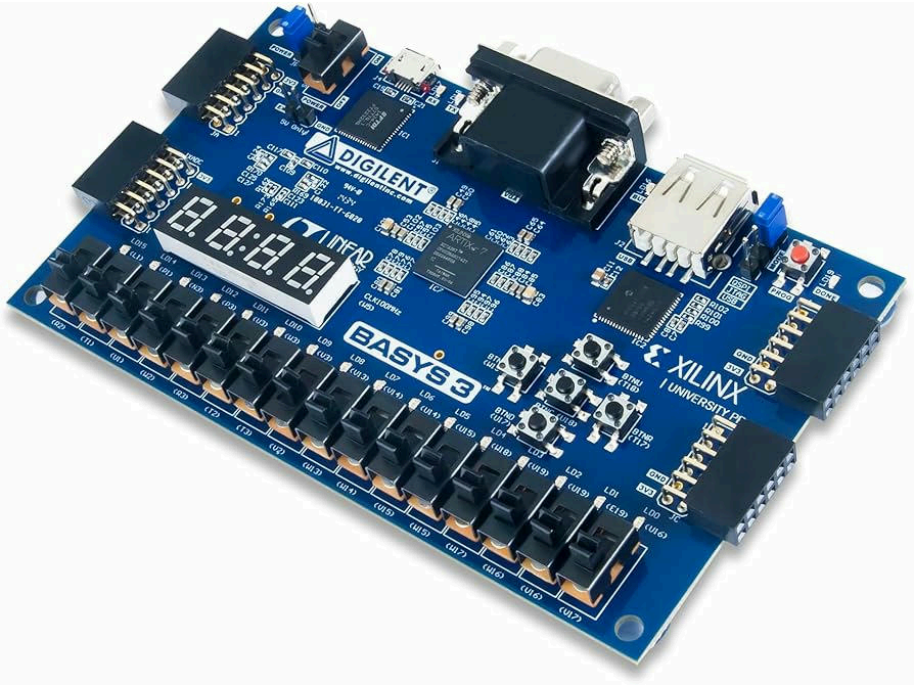
- Sistemas Heterogéneos.
  - CPU
  - Coprocesador.
- Detalles dependen del tipo de Coprocesador.
- Madurez de herramienta.
- Rendimiento práctico mucho menor que teórico.

Tipos de Coprocesadores

- Targetas de video : GPU.
  - Cómputo de alto rendimiento aprovecha mercado de video de alta calidad.
  - Entrenamiento Deep Learning.
- FPGA.
  - Circuitos programables.
  - Rápido y flexible.
  - Más lejos del diseño de un CPU genérico.

FPGA (Field Programmable Gate Array)

- Circuito integrado semiconductor reprogramable que permite configurar la lógica digital interna después de su fabricación
- Uso en electrónica programable.
- Diferentes fabricantes:
  - Altera
    - Intel compró Altera en 2015
  - Xylinx.
- Programación tipicamente en Verilog.
- Existen compiladores para HPC para C y Fortran.
- **Usado para resolver Modelos de Equilibrio General**



Programming FPGAs for Economics : An Introduction to Electrical Engineering Economics (Cheela et al., 2025)

QUANTITATIVE ECONOMICS
JOURNAL OF THE ECONOMETRIC SOCIETY

Original Articles | Open Access

Programming FPGAs for economics: An introduction to electrical engineering economics

Bhagath Cheela, André DeHon, Jesús Fernández-Villaverde, Alessandro Peri

First published: 03 February 2025 | https://doi.org/10.3982/QE2344 | Citations: 1

We thank Yicheng Li (UPenn, Engineering) for the initial implementation of our hardware design. We thank Lucas Ladenburger, Marina Leah McCann, and Paro Suh (CU Boulder, Economics) for outstanding research assistance. We thank Andrew Monaghan and the CU Boulder Research Computing Center for providing valuable insights. We also thank Giuseppe Bruno and Riccardo Russo (Bank of Italy) for testing an early version of the tutorial associated with this paper and Victor Duarte, Mahdi E. Kahou, and Jesse Perla for their comments. This project used: (i) the RMACC Summit supercomputer, supported by the National Science Foundation (awards ACI-1532235 and ACI-1532236), CU Boulder and Colorado State University; (ii) AWS Credits awarded under the NSF CC* Hybrid Cloud Award OAC-1925766, Research Computing, CU Boulder, 2022. This project was also supported by the Undergraduate Research Experiences for Diversity Grant, 2021, Institute of Behavioral Science, University of Colorado, USA.

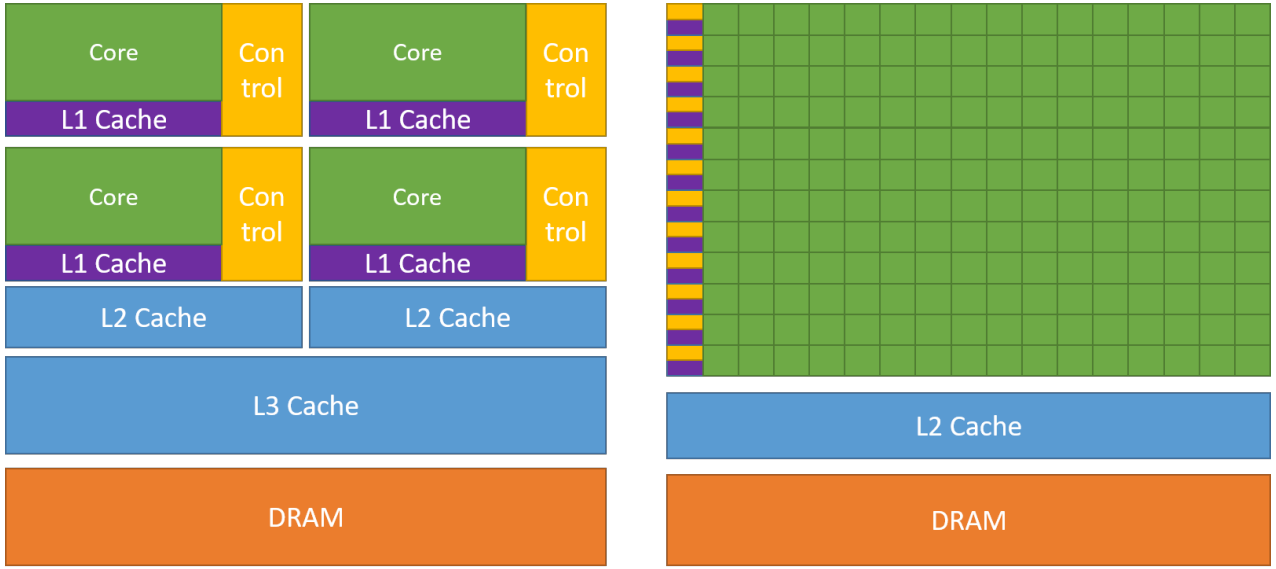
GPU (Graphics processing unit)

- Mercado dominado por NVIDIA.
- Arquitecturas : Tesla, Fermi, Kepler, Maxwell, Pascal, Volta, Turing, etc.
- **CUDA** : introducida 2007, con tesla.
- Parecido a C.
- Es posible programar GPU con **OpenMP** siguiendo el modelo fork-join.
- Problema : Portabilidad.
- **Mojo**!!

Mojo is blazing fast

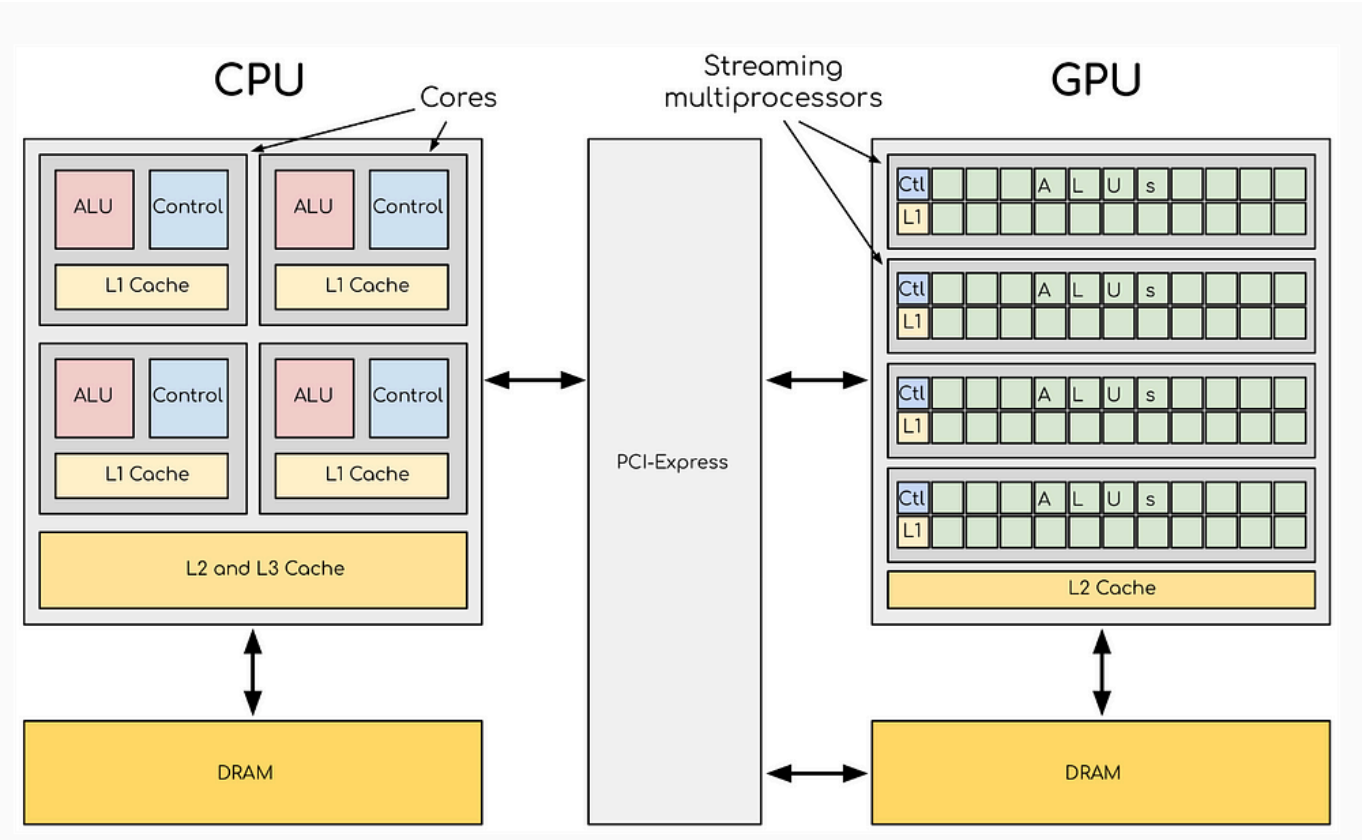
- **100-1000X faster** than Python.
- Full power of modern CPUs.
  - SIMD, threads, NUMA, etc.
  - New MLIR-based compiler.
- Works on GPU too.
  - **Same Language** for both CPU and GPU.
- Power of Pythonic Mojo code.
  - Easy to learn.
  - Great for low level performance.
  - Powerful metaprogramming.

CPU, diseño orientado a latencia, GPU, diseño orientado a throughput[1].



[1] The NVIDIA H100, for example, can run 16,896 threads simultaneously in a single clock cycle, with over 270,000 threads queued for execution.

Cómputo Heterogéneo



Conceptos clave

- Kernel.
- Thread / Block / Grid
- Host / Device Memory
- Global / Shared Memory

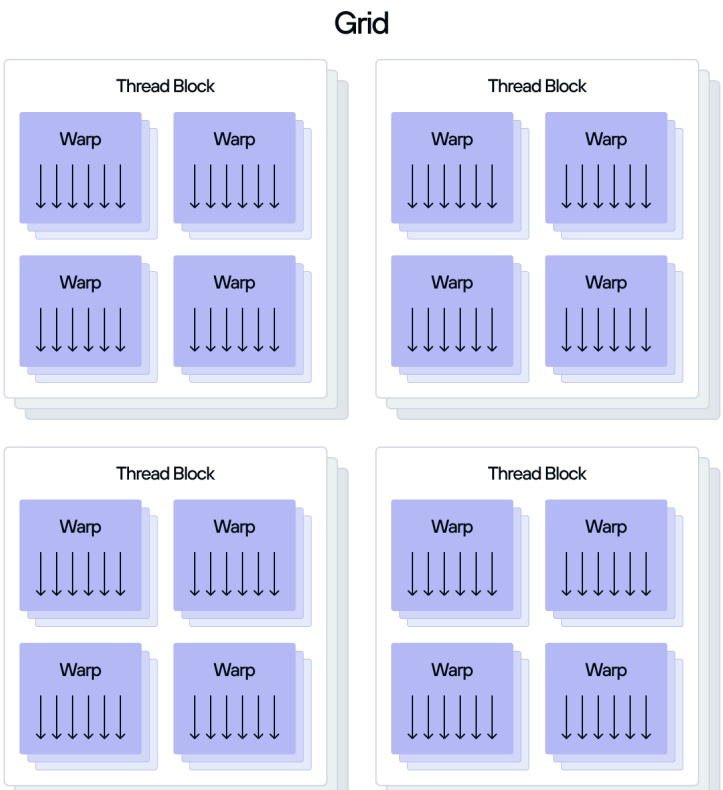
Kernel

Kernel function o GPU Kernel es simplemente una función que se ejecuta en la GPU, ejecutando un cálculo específico en un conjunto grande de datos en paralelo mediante miles o millones de threads.

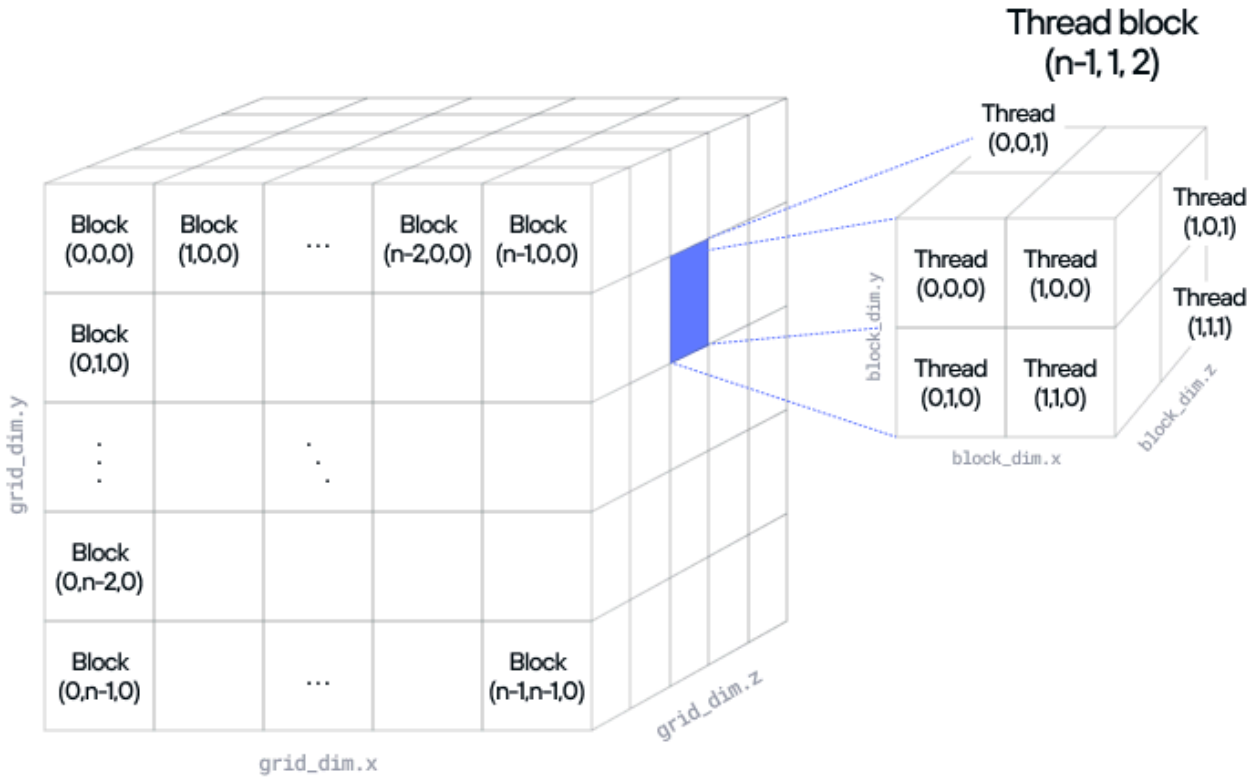
```python
from std.gpu import block_dim, block_idx, thread_idx
from std.math import ceildiv
# Calculate the number of thread blocks needed by dividing the vector size
# by the block size and rounding up.
comptime block_size = 256
comptime num_blocks = ceildiv(vector_size, block_size)


def vector_addition(
    lhs_tensor: LayoutTensor[float_dtype, layout, MutAnyOrigin],
    rhs_tensor: LayoutTensor[float_dtype, layout, MutAnyOrigin],
    out_tensor: LayoutTensor[float_dtype, layout, MutAnyOrigin],
):
    """Calculate the element-wise sum of two vectors on the GPU."""

    # Calculate the index of the vector element for the thread to process
    var tid = block_idx.x * block_dim.x + thread_idx.x

    # Don't process out of bounds elements
    if tid < vector_size:
        out_tensor[tid] = lhs_tensor[tid] + rhs_tensor[tid]
```

Thread

- Unidad básica de una ejecución paralela en una GPU.
- Cada thread corre el mismo kernel function pero procesa diferentes piezas de datos.
- Un **thread block** es un grupo cordinado de threads que pueden compartir memoria y sincronizarse entre sí.
- Thread dentro de un bloque pueden comunicarse muy eficientemente.
- Un **grid** es el nivel top de la estructura organizacional para la ejecución del kernel function.
- Un grid consiste de múltiples bloques de threads que se dividen aún más en threads individuales que ejecutan el kernel function concurrentemente.



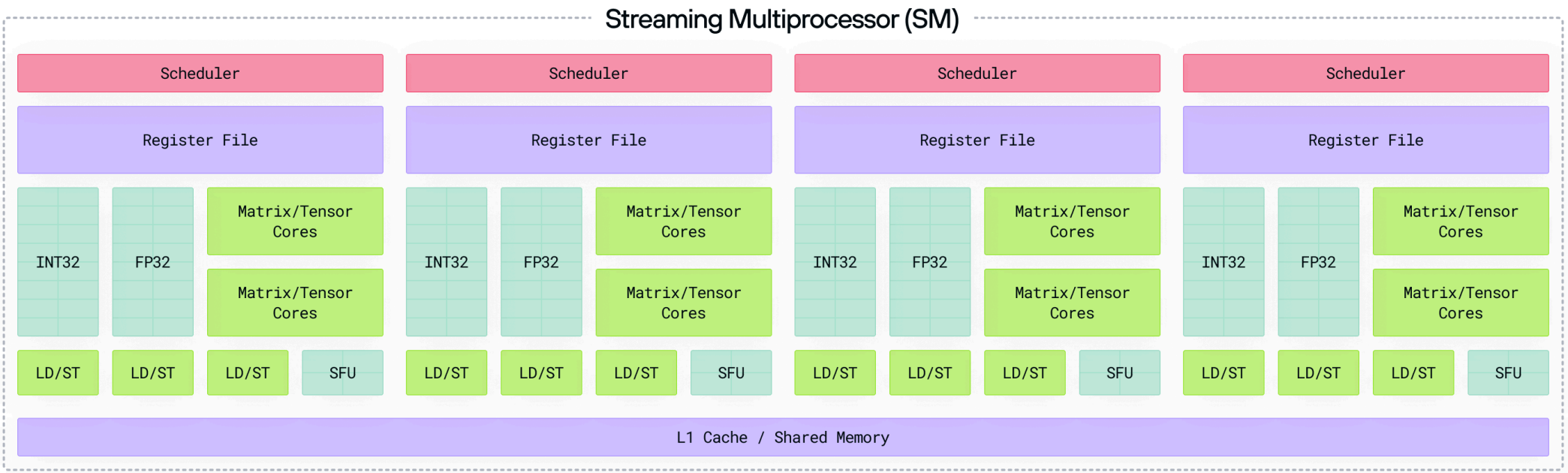
Organization of thread blocks and threads within a grid.



Global Memory y Shared Memory

- **Global Memory** es el espacio de memoria principal de la GPU accesible para todos los threads.
- El grande, pero relativamente lenta.
- **Shared Memory** es una memoria rápida onchip que los thread en un bloque usan para comunicarse.
- Es varios órdenes de magnitud más rápida que la memoria global, lo que la hace crítica para el desempeño.

Shared Memory

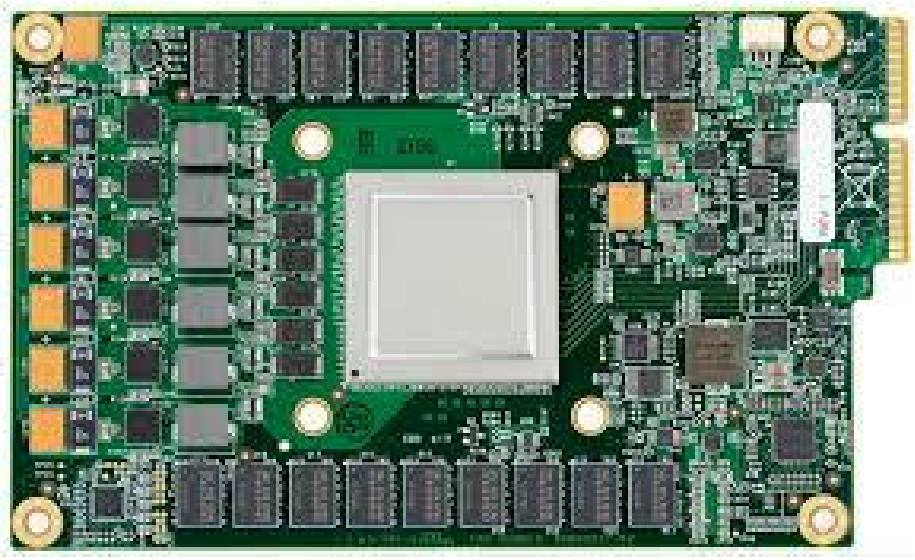


Aceleradores para Machine Learning y Deep Learning

- Intensivos en Álgebra Lineal.
- Productos para:
  - Equipos portátiles.
  - Sistemas embebidos (e.g coches)

TPU (Google Tensor Processing Unit)

- Para acelerar TensorFlow y **JAX**
- Acelerar el proceso de Entrenamiento.



JAX : A flexible and fast opensource numerical computing library from Google DeepMind

- JAX proporciona una API estilo NumPy para la adopción fácil de ideas o pruebas de concepto en Cómputo Científico o Machine Learning.
- Obtenemos un mayor speed-up simplemente sustituyendo NumPy por JAX NumPy.

```python
import jax.numpy as jnp

def predict(params, inputs):
  for W,b in params:
    outputs = jnp.dot(inputs, W) + b
    inputs = jnp.tanh(outputs)

  return outputs

def loss(params, batch):
  inputs, targets = batch
  preds = predict(params, inputs)

  return jnp.sum((preds - targets)**2)
```

JAX Function Transformation : Automatic Differentation.

- Gradient Transformation : JAX toma cualquier función como input y la transforma en una función que calcula el gradiente de la función original.
- Esto lo logra mediante **Diferenciación Automática**, comunmente llamada **autodiff**, una herramienta crucial en Machine Learning y tareas de optimización.
- En este caso transformamos la función de pérdida con `grad` y esto nos permitiría calcular el gradiente de nuestra función de pérdida y ajustar los pesos de nuestro modelo para minimizar el error.

Python

```python
import jax.numpy as jnp
from jax import grad

def predict(params, inputs):
  for W,b in params:
    outputs = jnp.dot(inputs, W) + b
    inputs = jnp.tanh(outputs)

  return outputs

def loss(params, batch):
  inputs, targets = batch
  preds = predict(params, inputs)

  return jnp.sum((preds - targets)**2)

gradient_fun = grad(loss)
```

JAX Function Transformation : Batching.

- La función `vmap` facilita la operación de tomar una función que fue escrita para un elemento en una función que funcione para muchos elementos, operación comunmente denominada como **vectorización** o **batching**.
- En este ejemplo vectorizamos la función que calcula el gradiente de nuestra función de pérdida usando `vmap`, permitiendo calcular los gradientes para múltiples elementos.

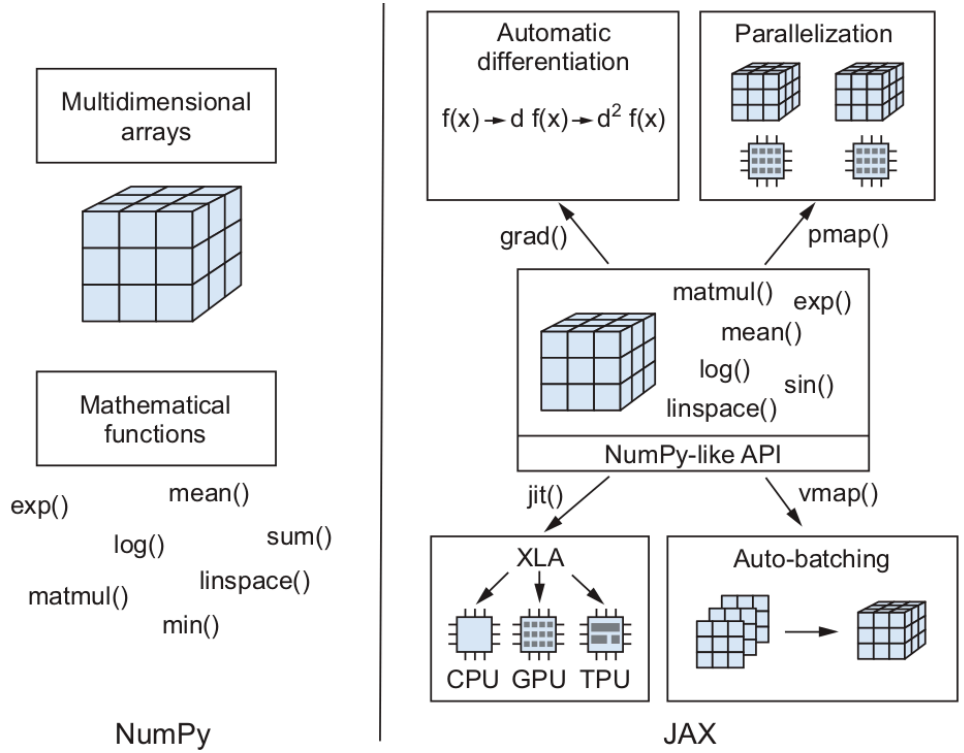
```python
import jax.numpy as jnp
from jax import grad, vmap

def predict(params, inputs):
  for W,b in params:
    outputs = jnp.dot(inputs, W) + b
    inputs = jnp.tanh(outputs)

  return outputs

def loss(params, batch):
  inputs, targets = batch
  preds = predict(params, inputs)

  return jnp.sum((preds - targets)**2)

gradient_fun = grad(loss)
perexample_grads = vmap(grad(loss), in_axes = (None,0))
```

JAX Function Transformation : Compilation.

- ¿JAX cómo acelera el procesamiento? JAX puede ejecutar compilación Just-In-Time, JIT, para crear código optimizado que pueda ejecutarse rapidamente.
- La compilación permite reducir la latencia y usar nuestro hardware eficientemente de forma automática, eliminando salidas no usadas y recomputaciones innecesarias.}
- También podemos compilar las operaciones de diferenciación automática.

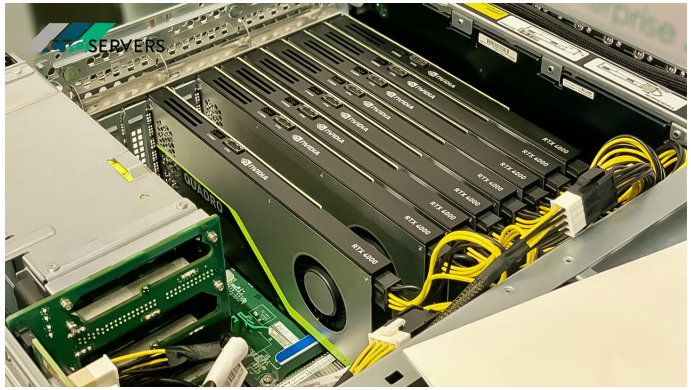
```python
import jax.numpy as jnp
from jax import grad, vmap, jit

def predict(params, inputs):
  for W,b in params:
    outputs = jnp.dot(inputs, W) + b
    inputs = jnp.tanh(outputs)

  return outputs

def loss(params, batch):
  inputs, targets = batch
  preds = predict(params, inputs)

  return jnp.sum((preds - targets)**2)

gradient_fun = jit(grad(loss))
perexample_grads = jit(vmap(grad(loss), in_axes = (None,0)))
```

JAX : A flexible and fast numerical computing library



JAX provides a set of composable function transformations for Python + NumPy code for compilation, vectorization, parallelization, and automatic differentiation. Source : (Sapunov, 2024)

JAX : Run anywhere and scale automatically



Any hardware: CPU, GPU (NVIDIA & AMD), TPU

Unified API: Any scale

Evaluemos JAX usando CPU y GPU 🤓 🖥️

🐍 Python

```python
import numpy as np
import jax.numpy as jnp
from jax import device_put, devices, jit
# a function with some amount of calculations
def f(x):
  y1 = x + x*x + 3
  y2 = x*x + x*x.T
  return y1*y2
# generate some random data
x = np.random.randn(3_000, 3_000).astype('float32')
jax_x_gpu = device_put(jnp.array(x), devices("gpu")[0])
jax_x_cpu = device_put(jnp.array(x), devices("cpu")[0])
# compile function to CPU and GPU backends with JAX
jax_f_gpu = jit(f, backend = "gpu")
jax_f_cpu = jit(f, backend = "cpu")
# warm-up
jax_f_cpu(jax_x_cpu)
jax_f_gpu(jax_x_gpu)
```

JAX para solucionar problema de optimización intertemporal de los hogares (Fehr & Kindermann, 2018) 🤓 🖥️

Considera el siguiente problema de optimización intertemporal de los hogares: La función de utilidad de los hogares está dada por

$$U(c_1, c_2, c_3) = \sum_{i=1}^{3} \beta^{i-1} u(c_i) \qquad \text{con} \qquad u(c_i) = \frac{c_i^{1-\frac{1}{\gamma}}}{1-\frac{1}{\gamma}}$$

donde c_i define el consumo en el periodo i de vida, β denota la tasa de descuento y γ es la elasticidad de sustitución intertemporal.

Asume que los hogares reciben ingreso laboral w en los primeros dos periodos y consumen todos sus ahorros en el tercer periodo, de manera que la restricción presupuestaria queda definida como:

$$\sum_{i=1}^{3} \frac{c_i}{(1+r)^{i-1}} = \sum_{i=1}^{2} \frac{w}{(1+r)^{i-1}}$$

donde r es la tasa de interés.

Resuelve los niveles de consumo óptimo usando algún método de optimización de primer orden.

Procede de la siguiente manera:

- a) Sustituye la restricción presupuestaria en la función de utilidad de manera que esta dependa únicamente de c_2 y c_3.
- b) Minimiza la función $-\tilde{U}(c_2, c_3)$ con el objetivo de obtener los valores óptimos de c_2 y c_3 (c_2^* y c_3^*).
- c) Deriva c_1^* de la restricción presupuestaria.

Usa los siguientes parámetros: $\boldsymbol{\gamma = 0.5}$ $\beta = 1$

Realiza el siguiente análisis de sensibilidad variando el valor de w y r de acuerdo a la siguiente tabla:

w	r	c_1	c_2	c_3
1	0	0.66	0.66	0.66
2	0	1.33	1.33	1.33
1	0.1	0.66	0.69	0.73

Los valores de c_1, c_2 y c_3 son los obtenidos por Fehr y Kindermann (2018).

Bibliography

Cheela, B., DeHon, A., Fernández-Villaverde, J., & Peri, A. (2025). Programming FPGAs for economics: An introduction to electrical engineering economics. *Quantitative Economics*, *16*(1), 49–87.

Fehr, H., & Kindermann, F. (2018). *Introduction to computational economics using Fortran*. Oxford University Press.

Sapunov, G. (2024). *Deep learning with JAX*. Manning.